Safe Learning of Adaptive Control Policies for Remote Patient Monitoring

Ramanan Tamizholi[1], Siddharth Chandak[2], Isha Thapa[3], Nicholas Bambos[2,3] and David Scheinker[4]

***Abstract*— Remote Patient Monitoring (RPM) enables continuous observation of patients in their daily environments, improving both health outcomes and quality of life. A key challenge in RPM is determining the optimal monitoring intensity, while balancing patient safety and monitoring costs. This problem is further complicated when system parameters, such as transition probabilities and costs, are initially unknown. We develop a learning-based control framework that estimates these parameters and adapts the monitoring policy in real time. The proposed approach is a novel online model-based reinforcement learning algorithm tailored to RPM, with patient safety explicitly prioritized during exploration. We provide theoretical guarantees on safety and convergence to the optimal policy. Simulation results show that the algorithm converges to the optimal threshold-based policy, maintains low treatment costs, and reduces the risk of patients reaching critical health states.**

I. Introduction

Remote Patient Monitoring (RPM) is becoming an increasingly integral part of modern healthcare, enabling continuous observation of patients within their daily environments and enhancing both their quality of life and healthcare outcomes [1]–[3]. Advances in wearable medical devices, such as continuous glucose monitors [4] and smartwatches with vital sign monitoring capabilities [5] among others have facilitated the collection and transmission of extensive health data for real-time analysis and timely medical interventions.

A significant challenge in RPM is determining the optimal intensity of patient monitoring. In clinical settings, adjusting monitoring intensity may involve increasing the frequency of data review, initiating outreach to patients, or escalating to in-person visits, significantly impacting provider workload. From the patient's perspective, while intensive monitoring can lead to earlier detection of health issues and prompt responses, it may also cause treatment burden or fatigue due to its invasive nature [6]. Conversely, less intensive monitoring is less intrusive to the patient, but might not provide sufficient patient support. This makes the decision of when and how intensively to monitor a key operational and clinical consideration, one that requires balancing health outcomes and monitoring costs by dynamically adapting the monitoring intensity to the patient's health state.

Recent works [7], [8] addressed this trade-off by modeling the RPM service as a Markov Decision Process (MDP), and showed that *threshold policies*, where patients switch to intensive monitoring once their health falls below a certain threshold are optimal. These analyses characterized how the optimal policy depends on key parameters such as health improvement probabilities, monitoring options, and invasiveness costs, enabling the framework to be adapted to specific medical contexts and individualized patient needs. However, these works assumed that the transition probabilities and patient costs in the MDP are known a priori to clinicians.

In practice, parameters may be unknown at the time of service deployment. Newer interventions may lack historical data, while established programs may encounter changing clinical guidelines for how health metrics are defined and prioritized [9]. RPM systems naturally generate high-frequency patient data streams, creating the opportunity to adapt monitoring strategies in response to observed patient outcomes during deployment. However, doing so safely requires balancing the need to explore untested actions with the goal of consistently delivering high-quality care.

To address this challenge, we formulate the problem as a Reinforcement Learning (RL) task [10]. Most works applying RL in healthcare focus on optimizing treatment strategies, such as medication timing and dosing [11]–[13]. In contrast, we address the monitoring control problem: learning when to intensively observe patients. Rather than automating treatment decisions, we develop an interpretable support tool that helps clinicians allocate attention effectively.

We propose an algorithm that simultaneously learns the system parameters and adapts the monitoring policy in real time. Since learning occurs while the RPM service is deployed on patients, exploration must be carried out without compromising patient safety or imposing excessive monitoring costs. To address this, we introduce a simple randomized exploration scheme tailored to the tiered RPM setting. By occasionally adjusting the implemented threshold toward clinically meaningful health levels, the algorithm promotes exploration while preserving patient safety. We show that, with appropriate initialization, the algorithm satisfies safety guarantees during learning and that the resulting policy converges to the optimal threshold policy under mild assumptions. Simulations further demonstrate that the learned policy converges to the optimal threshold control while maintaining low costs and reducing the risk of patients reaching critical health states.

Our framework offers a practical approach to adaptive monitoring in patient care, addressing a core yet understudied dimension of clinical decision-making: when and how intensively to observe patients. By learning system dynamics

[1] Indian Institute of Science (IISc), Bangalore, India.
[2] Department of Electrical Engineering, Stanford University, USA.
[3] Department of Management Science & Engineering, Stanford University, USA.
[4] School of Medicine, Stanford University, USA.
Emails: ramanant@iisc.ac.in, {chandaks, ishadt, bambos, dscheink}@stanford.edu

during deployment, our method enables real-time adaptation to evolving patient data, institutional constraints, and clinical guidelines without assuming prior knowledge. The resulting threshold policies are transparent and interpretable, providing clinicians with actionable guidance while preserving human oversight. These features make our approach a promising step toward more flexible and deployable RPM tools.

II. The Remote Patient Monitoring Model

Consider a patient who can be in a **health state** $h_t \in \{0, 1, 2, 3, ..., H\}$ in each service time period $t \in \{0, 1, 2, 3, ...\}$. The RPM service places the patient in a monitoring/intervention state $m_t \in \mathcal{M} = \{o, i\}$ in each time period t, abbreviated to **monitoring state**, where o denotes ordinary monitoring and i intensive monitoring. Thus, one can view the monitoring-patient joint state $s_t := (m_t, h_t) \in \mathcal{M} \times \mathcal{H} =: \mathcal{S}$ as the system or service state at time t.

A higher patient health state $h \in \{0, 1, 2, 3, ..., H\}$ corresponds to the patient having better health. In particular, the lowest health state 0 is **critical** in the sense that, when the patient drifts into that state under any monitoring state i or o, they go beyond the scope of the current service model and the service evolution stops. At that point other emergency and/or more severe medical interventions are required.

An advantage of our model is that the costs can be interpreted from multiple perspectives. But in this section, we first take the patient's treatment burden [6] point of view. Under ordinary monitoring, the patient incurs a constant cost $C_o \geq 0$ at any state (o, h) with $h \in \{1, 2, \ldots, H\}$. Correspondingly, under intensive monitoring, the patient incurs a constant cost $C_i \geq 0$ at any state (i, h). These costs reflect the invasiveness of the monitoring process to the patient's everyday lifestyle and quality of life. Of special interest are the critical health states, where this model ceases to apply. In such states, a significant cost of C_c is incurred.

A. Markov Decision Process

We model the system as a Markov Decision Process (MDP). Such models are commonly used for medical decision making [14], [15]. At the beginning of every time period t, the service decides whether to change the monitoring intensity. Formally, the decision/action space is $\mathcal{A} = \{o, i\}$ and each (state, action) pair is associated with a cost given by the function $c : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}^+$. The transition probabilities are given by $p(s'|s, a)$ where $s', s \in \mathcal{S}$ and $a \in \mathcal{A}$. The cost functions and transition probabilities are defined as follows, and have been illustrated in Figure 1a.

1. At health state $\mathbf{h = 0}$:

No action is taken and these states are absorbing states. A cost of C_c is incurred.

2. At health states $\mathbf{1 \leq h \leq H}$:

a) *Ordinary Monitoring ($m = o$), no Switching ($a = o$):* No monitoring change. The transition from (o, h) is:
   i) $(o, h) \xrightarrow{a=o} (o, \min\{h + 1, H\})$ with prob. λ_o
   ii) $(o, h) \xrightarrow{a=o} (o, h - 1)$ with prob. $\mu_o = 1 - \lambda_o$,
   and a cost C_o is incurred. Note that $\min\{h + 1, H\}$ above is used to account for $(o, H) \to (o, H)$ with prob. λ_o since H is the highest health state.

b) *Intensive Monitoring ($m = i$), no Switching ($a = i$):* No monitoring change. The transition from (i, h) is:
   i) $(i, h) \xrightarrow{a=i} (i, \min\{h + 1, H\})$ with prob. λ_i
   ii) $(i, h) \xrightarrow{a=i} (i, h - 1)$ with prob. $\mu_i = 1 - \lambda_i$,
   and a cost $c\big((i, h), i\big) = C_i$ is incurred.

c) *Intensive Monitoring ($m = i$) with Switching ($a = o$):* Induces a switch to ordinary monitoring. The next state with respective transition probabilities, and the cost incurred is same as part (a): ordinary monitoring ($m = o$) with no switching ($a = o$).

d) *Ordinary Monitoring (m=o), with Switching (a=i):* Induces a switch to intensive monitoring. The next state with respective transition probabilities, and the cost incurred is same as part (b): intensive monitoring ($m = i$) with no switching ($a = i$).

The decision to switch (or not) is made in the beginning of the timestep. As a result, the transition probabilities and costs are decided solely by the control and the next monitoring state. We make the following *natural* assumptions.

Assumption 1. *a) The transition probabilities satisfy:* $\lambda_i \geq \lambda_o$.
b) The costs satisfy: $0 \leq C_o \leq C_i \leq C_c$.

The first assumption 1.a) states that the patient's health improves faster under intensive monitoring, rather than under ordinary. Regarding assumption 1.b), it is naturally expected that $C_o \leq C_i$, as the treatment burden is higher under intensive monitoring. Further, given the severity of entering the critical state $h = 0$, it is naturally expected that $C_i \leq C_c$, and practically C_c is expected to be *much larger* than C_i.

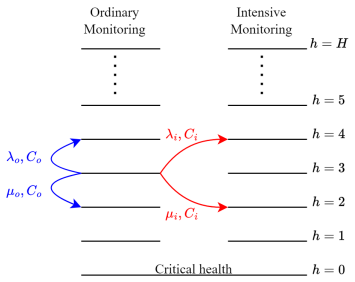
B. Optimal Monitoring Control

We study this problem under the *discounted cost* setting of the dynamic programming (DP) methodology [16]. Costs incurred t time periods into the future are discounted by a factor of γ^t with $0 < \gamma < 1$. Starting from state $s_0 = s \in \mathcal{S}$, the total expected (discounted) cost to be incurred is

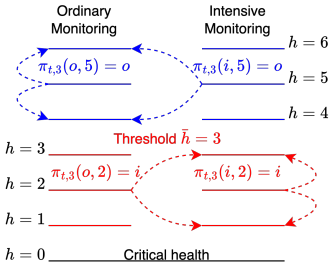
$$\mathbb{E}\Big[\sum_{t=0}^{T-1} \gamma^t c(s_t, a_t) + \gamma^T C_c \Big| \; s_o = s\Big]$$

At each time t, control action a_t is taken while in state $s_t = (m_t, h_t)$, at cost $c(s_t, a_t)$. The service ceases operation when the patient enters the critical state $h = 0$ at time T, where T is random and determined by evolution of the patient's health. At time T, the critical cost C_c is incurred, discounted to $\gamma^T C_c$. Thus, discounting by γ implicitly reflects the patient's desire to stay away from the critical health states.

A stationary monitoring policy π is a rule mapping each state $s \in \mathcal{S}$ to a control action $a \in \mathcal{A} = \{o, i\}$ to be taken at that state. The value function $V_\pi(s)$ of a policy π is the total expected discounted cost the patient will incur until reaching



(a) The Remote Patient Monitoring Model



(b) Threshold Policy $\pi_{t,\bar{h}}$ with $\bar{h} = 3$

Fig. 1: (a) The RPM service model. The blue and red arrows represent the possible transitions from state $(o, 3)$ for actions o and i, respectively. The arrows are labeled with probability of transition and cost incurred. (b) An example where the optimal policy is the threshold policy, i.e., ordinary monitoring is used for $h > 3$ and intensive for $h \leq 3$.

the critical state, when it starts from state s at time $t = 0$:

$$V_\pi(s) = \mathbb{E}\left[\sum_{t=0}^{T-1} \gamma^t c\Big(s_t, \pi(s_t)\Big) + \gamma^T C_c \;\Big|\; s_0 = s\right].$$

The goal is to find an optimal control π^* which minimizes V_π over all controls π, i.e., $V_{\pi^*}(s) \leq V_\pi(s)$ for every $s \in \mathcal{S}$ over all controls π. We define $V^*(s) := V_{\pi^*}(s)$ which satisfies the following dynamic programming equation [16].

$$V^*(s) = \min_{a\in\{o,i\}} \left\{ c(s,a) + \gamma \sum_{s'\in\mathcal{S}} \mathbb{P}\Big(s'|s,a\Big) V^*(s') \right\},$$

and can be solved numerically to yield the optimal policy $\pi^*(s) = \pi^*(m,h)$, that is, what optimal decision to take when the patient is in health state h under monitoring m. For our model, the dynamic programming equation is as follows:

(i) At the critical health state $h = 0$,

$$V^*(i,0) = V^*(o,0) = C_c, \tag{1}$$

(ii) For health states $1 \leq h \leq H$,

$$\begin{aligned} V^*(i,h) &= V^*(o,h) \\ &= \min\Bigg\{ C_i + \gamma\Big[\lambda_i V^*(i,h^+) + \mu_i V^*(i,h^-)\Big], \\ &\qquad C_o + \gamma\Big[\lambda_o V^*(o,h^+) + \mu_o V^*(o,h^-)\Big] \Bigg\}, \end{aligned} \tag{2}$$

where $h^+ = \min\{h+1, H\}$ and $h^- = h - 1$.

We next define a **threshold policy** $\pi_{t,\bar{h}}$, characterized by the threshold $\bar{h}$. Under these policies, there exists a threshold $\bar{h}$ such that the patient stays in ordinary monitoring only when their health is better than $\bar{h}$ (see Figure 1b for an example). So $\pi_{t,\bar{h}}(i,h) = \pi_{t,\bar{h}}(o,h) = i$ for $1 \leq h \leq \bar{h}$ and $\pi_{t,\bar{h}}(i,h) = \pi_{t,\bar{h}}(o,h) = o$ for $h > \bar{h}$. The following result makes an important observation about the optimal control.

Theorem 1. *Under the assumption that H and $1/\gamma$ are sufficiently large, the optimal policy π^* is a threshold-based policy $\pi_{t,\bar{h}}$ for some $0 \leq \bar{h} \leq H$.*

Proof. See Appendix I[1]. □

[1]Due to limited space, the Appendix has been uploaded to https://chandak1299.github.io/Appx_Learn_RPM.pdf

This result implies that we can reduce our problem to identifying optimal threshold $\bar{h}$ by restricting our attention to policies of the form $\pi_{t,\bar{h}}$. This result has been further corroborated by numerical experiments in [7], [8].

III. Learning the Unknown RPM Model

In the scenario where the parameters of the RPM service model are known to the clinicians, they can just obtain the optimal policy by solving the dynamic programming equation (2). But in reality, parameters such as the costs (C_o and C_i) and the probabilities (λ_o and λ_i) are not accurately known to the clinicians. A rough estimate of the parameters might be available based on similar implementations of RPM service, but it is necessary to learn the true parameters to achieve the optimal benefits from the RPM service.

The manner in which the samples are obtained for learning parameters depends on the exact service model and the definition of the parameters. If the cost parameters reflect the treatment burden on the patient, then the samples for estimating the invasiveness cost can be obtained by surveying patients about their experience with the monitoring service. In contrast, estimating the probability parameters is more straightforward, as it only requires observing changes in patient health state under different monitoring schemes. We assume that the critical cost is known to the service but it can be estimated in the same way as other costs.

It is important to note that these parameters are learned by observing the behavior of actual patients under the RPM service. We model this as an online reinforcement learning (RL) problem [10], where the control policy is continually updated based on the samples observed thus far. The monitoring control must be adjusted to enable efficient learning of the parameters while preventing patients from incurring excessive costs. We propose a novel exploration scheme in Algorithm 1, tailored to the RPM setting, that explicitly prioritizes patient safety during exploration.

The implementation of the algorithm is organized into epochs, each corresponding to a fixed time period (e.g., 10 days or one month). At the start of each epoch, the policy is computed based on the current parameter estimates and then applied to all patients throughout that epoch. During an epoch, both the patient population and its size may change, as new patients enroll in the remote monitoring program and

existing patients leave upon reaching the critical health state.

There are three parts to the algorithm. In the beginning of each epoch, the system parameters are updated based on the samples collected till now. Next, a policy is computed for the RPM service based on the parameter estimates. Finally, this control is used in remote monitoring of patients, and samples are obtained for cost and transition probabilities.

Algorithm 1 Online Model-Based Learning for RPM

Input: Priors $\lambda_{o,p}, \lambda_{i,p}, C_{o,p}, C_{i,p}$, prior strength n_0, healths h_{high} and h_{low}, probability sequence ρ_k and critical cost C_c

Initialization: Counts $N_a \leftarrow n_0$, $N_a^+ \leftarrow \lambda_{a,p} n_0$, cost sums $S_a \leftarrow C_{a,p} n_0$ for $a \in \{i, o\}$

for epoch $k = 1, \ldots,$ **do**

(1) Parameter Estimation:

for $a \in \{i, o\}$, compute estimates

$$\widehat{\lambda}_a \leftarrow N_a^+ / N_a \text{ and } \widehat{C}_a \leftarrow S_a / N_a$$

(2) Safe and Exploratory Monitoring Control

Compute optimal threshold based on current parameter estimates, denote by $\hat{h}_k$

Implemented threshold:

- if $h_{\text{low}} \leq \hat{h}_k \leq h_{\text{high}}$, $\tilde{h}_k = \hat{h}_k$,
- if $\hat{h}_k < h_{\text{low}}$, $\tilde{h}_k = \begin{cases} \hat{h}_k, & \text{w.p. } 1 - \rho_k, \\ h_{\text{low}}, & \text{w.p. } \rho_k. \end{cases}$
- if $\hat{h}_k > h_{\text{high}}$, $\tilde{h}_k = \begin{cases} \hat{h}_k, & \text{w.p. } 1 - \rho_k, \\ h_{\text{high}}, & \text{w.p. } \rho_k. \end{cases}$

(3) RPM Service and Data Collection:

for each patient in system **do**

for timesteps $t = 0, \ldots, T$ of epoch **do**

Observe health state h_t

Implement monitoring control $a = \pi_{t,\tilde{h}_k}(h_t)$

Observe next state h_{t+1} and cost sample c_t

Update:

$$N_a^+ \leftarrow N_a^+ + 1 \text{ if } h_{t+1} = h_t + 1$$
$$N_a \leftarrow N_a + 1, S_a \leftarrow S_a + c_t.$$

end for

end for

end for

Output: Final threshold $\hat{h}$ and estimates $\hat{\lambda}_a, \hat{C}_a$.

Before discussing the intuition behind Algorithm 1, we discuss some notation used therein. For $a \in \{i, o\}$, N_a denotes the number of times monitoring a has been applied, N_a^+ denotes the number of times the patient's health improved under monitoring a, and S_a denotes the total cost patients had to incur under monitoring a. These are used to compute estimate the transition probabilities $\widehat{\lambda}_a = N_a^+ / N_a$ and costs $\widehat{C}_a = S_a / N_a$. These estimates are initialized using the priors $\lambda_{a,p}$ and $C_{a,p}$ and the strength in these beliefs n_0. In each epoch, the optimal threshold corresponding to the current parameter estimates is computed and denoted by $\hat{h}_k$.

To ensure sufficient exploration in this tiered architecture, we adopt the following simple scheme. If the computed optimal threshold $\hat{h}_k$ is too low, the system may collect too few samples from patients receiving intensive monitoring. Therefore, when $\hat{h}_k < h_{\text{low}}$, we set the implemented threshold to h_{low} with probability ρ_k. Similarly, when $\hat{h}_k > h_{\text{high}}$, we set the implemented threshold to h_{high} with probability ρ_k. The probability sequence ρ_k is chosen to be slowly decreasing, so that the algorithm explores sufficiently in early epochs and gradually relies more on the estimated optimal threshold as the parameter estimates become more accurate.

This exploration scheme has three main advantages. First, it is intuitive, easy to implement, and tailored to the RPM setting. In particular, its parameters are clinically interpretable: h_{low} and h_{high} correspond to fragile and sufficiently good health states, while ρ_k can be chosen as any slowly decreasing sequence. Second, the scheme naturally supports safety guarantees. Third, under mild assumptions, the resulting policy converges almost surely to the optimal policy while incurring low regret. The following theorem establishes convergence under standard technical assumptions. The assumptions and proof are provided in Appendix II[2].

Theorem 2. *The policy (equivalently, the threshold) implemented by Algorithm 1 converges almost surely to the optimal policy (respectively, optimal threshold).*

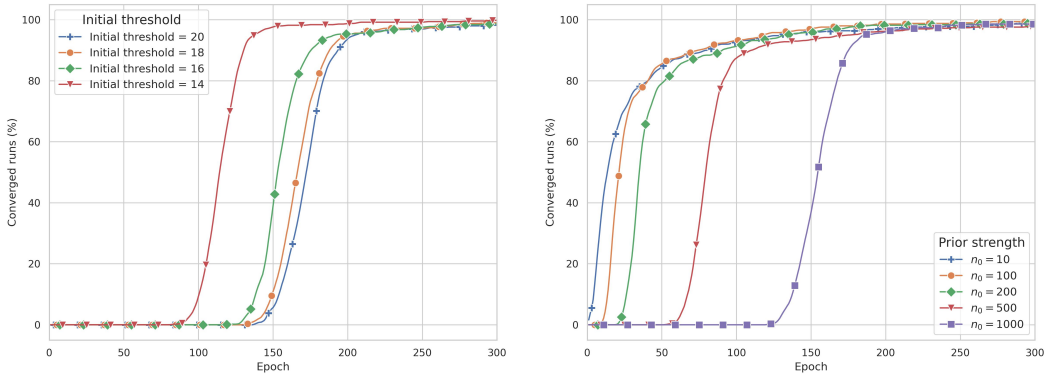
A. Safety Considerations

As the parameters are learned while the RPM service is deployed on real patients, it is essential that the policy avoid decisions that increase the risk of patients reaching critical health states. One extreme way to ensure safety is to place all patients under intensive monitoring at all health states. However, this slows parameter learning and substantially increases monitoring costs. With appropriate configuration, our algorithm balances safety and efficiency. In particular, the algorithm should be initialized by carefully choosing the priors $\lambda_{a,p}, C_{a,p}$ and the prior strength n_0.

The priors should be chosen to induce a conservative initial policy, since they determine the controls applied before sufficient data are collected. When rough parameter estimates are available, they can be used as priors. Otherwise, we recommend setting $\lambda_{i,p} \gg \lambda_{o,p}$ and $C_{i,p} \approx C_{o,p}$, which makes intensive monitoring more favorable, delaying the occurrence of critical states at the expense of higher monitoring cost. The prior strength n_0 controls how quickly the policy adapts: larger values slow parameter updates, prevent abrupt early policy shifts, and keep the policy close to its conservative initial form. The following theorem shows that if the priors are chosen as described above and the prior strength n_0 is sufficiently large, then, with high probability, the implemented threshold is never lower than the optimal threshold throughout the learning process. The assumptions and proof are provided in Appendix III[2].

Theorem 3. *For any $\delta \in (0, 1)$, if the prior strength satisfies $n_0 \geq \Omega(\log(1/\delta))$, then, with probability at least $1 - \delta$, the*

[2]Due to limited space, the Appendix has been uploaded to https://chandak1299.github.io/Appx_Learn_RPM.pdf



(a) Effect of initial threshold

(b) Effect of prior strength

Fig. 2: Percentage of converged runs after each epoch for Algorithm 1

implemented threshold satisfies $\tilde{h}_k \geq h^\star$ for all epochs k.

Beyond the guarantees of the theorem, the exploration mechanism itself provides an additional safeguard: when the estimated optimal threshold falls below h_{low}, the implemented threshold is raised to h_{low} with probability ρ_k, reducing the likelihood that the algorithm applies overly aggressive low-threshold policies during learning.

IV. Performance

In this section, we evaluate the performance and safety aspects of our algorithm through simulations. We examine its convergence to the optimal policy, and analyze how prior strength and initial threshold affect performance and safety.

To evaluate our algorithm, we simulate a Markov chain for each patient. Unless stated otherwise, each epoch of the algorithm consists of 75 timesteps distributed across patients. The plots of average time to reach the critical health state are obtained by averaging results over 200 independent runs. To plot the percentage of converged runs across epochs, we conduct 10 experiments, each comprising 50 runs. For each experiment, we compute the percentage of runs that have converged after every epoch and then average these percentages across all experiments. In all the plots the initial thresholds are chosen to be larger than the optimal threshold. A run is said to have converged if all estimated parameters are within 5% of their true values. Complete experimental details are provided in Appendix IV[3].

We observe that the algorithm converges to the optimal or a near-optimal policy within a few epochs in almost all runs. This behavior remains consistent across a wide range of parameter settings. In Figure 2, we plot the percentage of runs that have converged after each epoch. As expected, parameter estimates take longer to converge to the true values when the initial threshold is farther from the true threshold and when the prior strength is larger. However, this slower convergence of the parameter estimates has little practical impact, since the threshold policy is relatively robust to inaccuracies in the learned RPM parameters, allowing the algorithm to attain a near-optimal policy well before the underlying estimates have fully converged.

[3]Due to limited space, the Appendix has been uploaded to https://chandak1299.github.io/Appx_Learn_RPM.pdf

TABLE I: Table for safety of the implemented threshold for different prior strengths. Each entry reports the number of runs, out of 100 runs, in which the implemented threshold fell below the corresponding cutoff in at least once epoch.

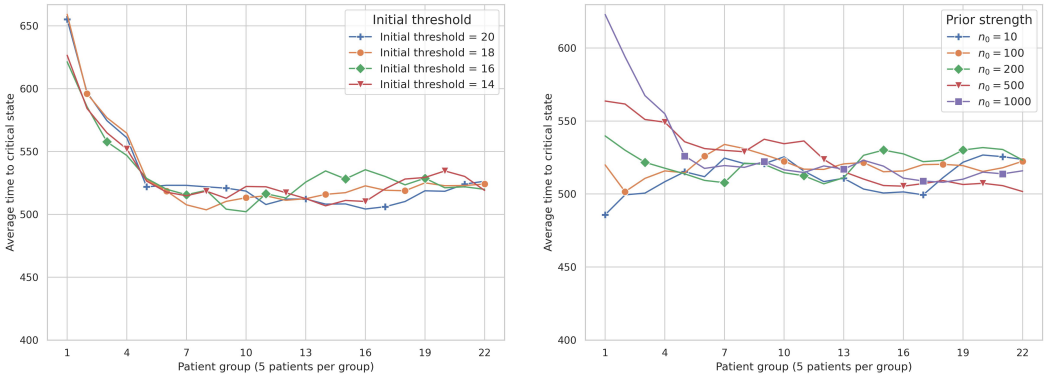
n_0	$h^\star$	h_{init}	$\tilde{h} < h^\star$	$\tilde{h} < h^\star - 1$	$\tilde{h} < h^\star - 2$	$\tilde{h} < h_{\text{frag}}$
10	12	16	82	38	8	0
100	12	16	50	1	0	0
200	12	16	25	0	0	0
500	12	16	6	0	0	0
1000	12	16	0	0	0	0

Figure 3 shows the average time to reach the critical health state under different initial thresholds and prior strengths. Across initial thresholds, the algorithm converges to the optimal policy ($\bar{h} = 12$) after approximately the same number of epochs, indicating robustness to initialization. The results also show that, for sufficiently large prior strengths, the average time to reach the critical health state remains consistently high throughout learning, indicating that the algorithm maintains patient safety while adapting the monitoring policy.

When the algorithm is initialized with a sufficiently conservative threshold and large prior strength, the implemented threshold remains above the optimal threshold in almost every epoch, as shown in Table I. This behavior is consistent with the safety guarantee in Theorem 3. We also report more severe threshold violations, where the implemented threshold falls below $h^\star - 1$, $h^\star - 2$, or h_{frag}. The number of such violations drops sharply as the severity of the violation increases. For example, across 100 independent simulation runs, even for a relatively small prior strength of $n_0 = 100$, the implemented threshold falls below $h^\star$ in 50 runs, but falls below $h^\star - 1$ in only one run and never falls below $h^\star - 2$ or h_{frag}. Increasing the prior strength further reduces the frequency of threshold violations, making the deployed policy increasingly conservative during the learning phase.

V. Conclusions and Extensions

We develop an online model-based reinforcement learning approach for optimizing RPM policies when system parameters are initially unknown. By combining parameter estimation with a simple exploration scheme, our algorithm



(a) Effect of initial threshold

(b) Effect of prior strength

Fig. 3: Average time to reach critical health for Algorithm 1. Patients are grouped only for reporting: group 1 contains the first five patients to enter the system, group 2 contains the next five, and so on. These groups do not represent synchronous cohorts, and patients within a group may enter and leave the system at different times.

efficiently balances learning and patient safety. Theoretical guarantees and simulations demonstrate that the proposed approach effectively learns the transition probabilities and costs, and converges to the optimal threshold policy. It maintains low overall costs and safe monitoring decisions, representing a promising step toward more flexible and adaptive RPM tools.

An important next step is to implement the proposed algorithm on real-world patient data to assess its practical performance. While our simulations demonstrate its effectiveness, testing on clinical data will reveal its robustness to real-world variability and noise. Such validation will be essential for integrating the approach into actual RPM workflows and guiding its deployment in clinical settings.

This paper focuses on the special case where RPM parameters are independent of health states. Although the proposed algorithm can be extended to health-state-dependent parameters, it is practical only for small state spaces, since the number of parameters and required samples grow quickly with the number and dimension of health states. Addressing this curse of dimensionality is an important future direction, potentially through approximate dynamic programming, deep reinforcement learning, or structural assumptions such as smoothness and parameter sharing across neighboring health states [10], [16]–[18].

References

[1] F. A. C. d. Farias, C. M. Dagostini, Y. d. A. Bicca, V. F. Falavigna, and A. Falavigna, "Remote patient monitoring: a systematic review," *Telemedicine and e-Health*, vol. 26, no. 5, pp. 576–583, 2020.

[2] L. P. Malasinghe, N. Ramzan, and K. Dahal, "Remote patient monitoring: a comprehensive study," *Journal of Ambient Intelligence and Humanized Computing*, vol. 10, pp. 57–76, 2019.

[3] A. Zinzuwadia, J. M. Goldberg, M. A. Hanson, and J. D. Wessler, "Continuous cardiology: the intersection of telehealth and remote patient monitoring," in *Emerging Practices in Telehealth*. Elsevier, 2023, pp. 97–115.

[4] I. Lee, D. Probst, D. Klonoff, and K. Sode, "Continuous glucose monitoring systems-current status and future perspectives of the flagship technologies in biosensor research," *Biosensors and Bioelectronics*, vol. 181, p. 113054, 2021.

[5] M. Masoumian Hosseini, S. T. Masoumian Hosseini, K. Qayumi, S. Hosseinzadeh, and S. S. Sajadi Tabar, "Smartwatches in healthcare medicine: assistance and monitoring; a scoping review," *BMC Medical Informatics and Decision Making*, vol. 23, no. 1, p. 248, 2023.

[6] B. W. Heckman, A. R. Mathew, and M. J. Carpenter, "Treatment burden and treatment fatigue as barriers to health," *Current opinion in psychology*, vol. 5, pp. 31–36, 2015.

[7] S. Chandak, I. Thapa, N. Bambos, and D. Scheinker, "Tiered service architecture for remote patient monitoring," in *2024 IEEE International Conference on E-health Networking, Application & Services (HealthCom)*, 2024, pp. 1–7.

[8] ——, "Optimal control for remote patient monitoring with multidimensional health states," in *ICC 2025 - IEEE International Conference on Communications*, 2025, pp. 3186–3192.

[9] I. Thapa, P.-A. Laforcade, F. K. Bishop, J. Ferstad, M. Desai, D. M. Maahs, P. Prahalad, D. P. Zaharieva, D. Scheinker, and R. Johari, "Regression discontinuity in time: Evaluating the impact of evolving digital health interventions," *International Journal of Medical Informatics*, vol. 204, p. 106050, 2025.

[10] R. S. Sutton, A. G. Barto *et al.*, *Reinforcement learning: An introduction*. MIT press Cambridge, 1998, vol. 1, no. 1.

[11] J. Watts, A. Khojandi, R. Vasudevan, and R. Ramdhani, "Optimizing individualized treatment planning for parkinson's disease using deep reinforcement learning," in *Proceedings of the Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, Jul 2020, pp. 5406–5409.

[12] A. Jafar, M. Tauschmann, T. Li, S. Schaller, L. Leelarathna, M. Wilinska, and R. Hovorka, "Personalized insulin dosing using reinforcement learning for high-fat meals and aerobic exercises in type 1 diabetes: A proof-of-concept trial," *Nature Communications*, vol. 15, no. 1, p. 6585, 2024.

[13] D. J. Luckett, E. Laber, A. E. Regh, M. R. Kidwell, and M. R. Kosorok, "Estimating dynamic treatment regimes in mobile health using v-learning," *Journal of the American Statistical Association*, vol. 115, no. 531, pp. 1279–1292, 2020.

[14] L. N. Steimle and B. T. Denton, *Markov Decision Processes for Screening and Treatment of Chronic Diseases*. Cham: Springer International Publishing, 2017, pp. 189–222.

[15] O. Alagoz, H. Hsu, A. J. Schaefer, and M. S. Roberts, "Markov decision processes: a tool for sequential decision making under uncertainty," *Medical Decision Making*, vol. 30, no. 4, pp. 474–483, 2010.

[16] D. Bertsekas, *Dynamic programming and optimal control*. Athena scientific, 2012, vol. II.

[17] W. B. Powell, *Approximate Dynamic Programming: Solving the curses of dimensionality*. John Wiley & Sons, 2007, vol. 703.

[18] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.